Sistema za upravljanje saobraćajem

Članovi tima
- Vladislav Radović SV 27/2021
- Nikola Mitrović SV 18/2021

Motivacija

Saobraćaj u modernim gradovima je postao složen i teško upravljiv zbog velikog broja vozila, gužvi, nesreća i nepredviđenih situacija. Trenutni sistemi za upravljanje saobraćajem uglavnom ne koriste dovoljno podataka kako bi regulisali saobraćaj optimalno.

Cilj projekta je razvoj inteligentnog sistema zasnovanog na znanju, koji automatski analizira podatke o saobraćaju, detektuje kritične situacije i predlaže optimalne odluke za regulaciju prometa u realnom vremenu.

Pregled problema

Gradovi se suočavaju sa problemima poput gužvi, nesreća i nepredviđenih situacija, dok postojeći sistemi za regulaciju saobraćaja uglavnom ne koriste dovoljno dostupnih podataka i nisu prilagodljivi promenama u realnom vremenu.

Neophodno je razviti sistem koji: - Prikuplja i analizira podatke iz senzora, kamera i GPS uređaja u realnom vremenu - Otkriva kritične situacije i donosi optimalne odluke - Obezbeđuje validaciju podataka i konzistentnost stanja sistema - Omogućava obradu Complex Event Processing (CEP) za detekciju vremenskih obrazaca - Koristi perzistentnu sesiju za kontinuirano praćenje stanja saobraćaja

Metodologija rada

Sistemska arhitektura

Sistem je implementiran kao **Spring Boot** aplikacija koja koristi **Drools** engine za upravljanje pravilima i **Angular** frontend za interakciju sa korisnicima.

Ključne komponente: - **ValidationService** - Validacija crossroad referenci pre inserovanja faktova - **RuleExecutionService** - Perzistentna KieSession za kontinuirano upravljanje stanjem - **KieContainer** - Upravljanje Drools pravilima i CEP funkcionalnostima - **Angular Frontend** - Interaktivni UI sa realtime prikazom rezultata

Ulaz u sistem

Ulazi uključuju: - **TrafficDensity**(crossroad, value) - Gustina saobraćaja na raskrsnici - **Vehicle**(crossroad, speed, type) - Podaci o vozilima - **Accident**(crossroad, severity) - Informacije o nesrećama - **Weather**(type, intensity)

- Meteorološki uslovi - **EmergencyVehicle**(location, direction) - Hitna vozila - **IllegalParking**(crossroad) - Nepropisno parkiranje - **EventDay**(eventType, expectedTraffic) - Specijalni događaji - **PedestrianDetected**(crossroad, count) - Detekcija pešaka - **PublicTransportDelay**(line, delayMinutes) - Kašnjenje javnog prevoza - **Crossroad**(id, connectedRoads) - Definicija raskrsnice

Validacija podataka

Sistem implementira **striktnu validaciju crossroad referenci**:

```
@Service
public class ValidationService {
    public ValidationResult validateFactPayload(String factType,
                                                Map<String, Object> payload,
                                                List<Object> sessionFacts) {
        // Validacija da crossroad postoji pre kreiranja faktova
        if (factType.equals("Accident")) {
            String crossroadId = (String) payload.get("crossroad");
            if (!validateCrossroadExists(crossroadId, sessionFacts)) {
                return ValidationResult.invalid(
                    "Crossroad '" + crossroadId + "' does not exist in current session"
                );
            }
        }
        // Slično za IllegalParking, PedestrianDetected, TrafficDensity...
    }
}
```

Rezultat validacije: - **Validni faktovi** se inseruju u KieSession - **Nevalidni faktovi** se odbacuju sa detaljnim error porukama - **ValidationError** objekti se vraćaju korisniku

Perzistentna sesija

Sistem koristi **jednu KieSession** tokom životnog ciklusa aplikacije:

```
@Service
public class RuleExecutionService {
    private final KieSession persistentSession;

    @PostConstruct
    public void initSession() {
        persistentSession = kieContainer.newKieSession("ksession-rules");
        persistentSession.setGlobal("visitedCrossroads", new HashSet<String>());
    }

    @PreDestroy
    public void disposeSession() {
```

```
        if (persistentSession != null) {
            persistentSession.dispose();
        }
    }
}
```

Prednosti: - Kontinuirano praćenje stanja saobraćaja - Akumulacija faktova kroz vreme - CEP analiza vremenskih sekvenci - Bolja performance bez ponovnog kreiranja sesije

Baza znanja

Baza znanja je strukturisana po kontekstima i pravila su grupisana prema vremenskim periodima i tipovima događaja. **Sva pravila sada uključuju validaciju crossroad referenci** pre izvršavanja.

Primer konteksta i pravila:

1. Visoka gustina saobraćaja - produži zeleno

```
rule "High traffic density - extend the green light and find cause"
when
    $t : TrafficDensity($c : crossroad, value > 70)
    exists Crossroad(id == $c)  // Validacija da crossroad postoji
then
    insert(new TrafficAction($c, "extendGreen", 30));
    Notification n1 = new Notification("High traffic density: extended green light on " + $c
    insert(n1);
    insert(new CauseFact($c, "high_traffic_density"));
end
```

2. Detekcija nesreće - blokada raskrsnice

```
rule "Accident detected - block crossroad"
when
    Accident($c : crossroad, severity == "high")
    exists Crossroad(id == $c)  // Validacija da crossroad postoji
then
    insert(new TrafficAction($c, "blockCrossroad"));
    Notification n2 = new Notification("Accident on " + $c + " - crossroad is blocked");
    insert(n2);
end
```

3. Prioritet za hitna vozila

```
rule "Emergency vehicle priority"
when
    EmergencyVehicle($c : location, $d : direction)
```

```
    exists Crossroad(id == $c)  // Validacija da crossroad postoji
then
    insert(new TrafficAction($c, "allowPass", "priority"));
    Notification n = new Notification("Priority for emergency vehicle on crossroad " + $c);
    insert(n);
end
```

4. Loše vreme - produži zeleno

```
rule "Bad weather extend green"
when
    Weather(type == "rain" || type == "snow",
            intensity == "medium" || intensity == "high")
then
    insert(new TrafficAction("All", "extendGreen", 15));
    Notification n = new Notification("Bad weather, extended green light on crossroads");
    insert(n);
end
```

5. Pešaci na prelazu - crveno za vozila

```
rule "Pedestrian crossing - red light"
when
    PedestrianDetected($c : crossroad)
    exists Crossroad(id == $c)  // Validacija da crossroad postoji
then
    insert(new TrafficAction($c, "setRedForVehicles"));
    Notification n3 = new Notification("Pedestrian detected on " + $c);
    insert(n3);
end
```

6. Kašnjenje javnog prevoza - prioritet autobusu

```
rule "Bus delay - priority"
when
    PublicTransportDelay($line : line, delayMinutes > 10)
then
    insert(new TrafficAction("All", "prioritizeBus", $line));
    Notification n = new Notification("Autobus " + $line + " is delayed, give the priority c
    insert(n);
end
```

Nivoi ulančavanja pravila:

- **Prvi nivo** – detekcija osnovnog događaja (npr. gužva, nesreća, kašnjenje autobusa)

- **Drugi nivo** – aktivacija pravila koja kombinuju više ulaza i analiziraju uzroke
- **Treći nivo** – generisanje optimizovanih odluka i akcija

Primer ulančavanja pravila:

1. TrafficDensity $> 70\% \rightarrow$ aktivira pravilo za produžavanje zelenog
2. TrafficDensity + Accident $\rightarrow$ aktivira CEP pravilo za kritičnu situaciju
3. CriticalSituation $\rightarrow$ generiše TrafficAction i Notification objekte

Backward chaining i dijagnoza problema

Sistem koristi **backward chaining** i **rekurzivne querije** za identifikaciju uzroka problema sa naprednim mehanizmom za **prevenciju ciklusa**:

Globalna varijabla za praćenje posećenih raskrsnica

```
global java.util.Set visitedCrossroads;
```

Pravila za upravljanje stanjem rekurzije

```
// Pravilo za čišćenje posećenih raskrsnica na početku
rule "Clear visited crossroads"
when
then
    visitedCrossroads.clear();
end

// Pravilo za označavanje raskrsnice kao posećene
rule "Mark crossroad as visited"
when
    $cf : CauseFact($cr : crossroad)
then
    visitedCrossroads.add($cr);
end
```

Glavni rekurzivni query - findCauseOfDensity

```
query "findCauseOfDensity"(String target, String result)
    (
        // Base case: Direktan uzrok na trenutnoj raskrsnici
        $cf : CauseFact(crossroad == target)
        and
        eval(result == $cf.getCause())
    ) or (
        // Recursive case: Pretraži kroz povezane raskrsnice koje nisu posećene
        Crossroad(id == target, $connections : connectedRoads)
```

```
        and
        String($next : this) from $connections
        and
        // Proveri da raskrsnica nije već posećena (prevencija ciklusa)
        eval(!visitedCrossroads.contains($next))
        and
        // Označi kao posećenu pre rekurzije
        eval(visitedCrossroads.add($next))
        and
        findCauseOfDensity($next, result;)  // Rekurzivni poziv
    )
end
```

Specifični queriji za dijagnozu:

Direktan uzrok

```
query "getCause"(String crossroad, String cause)
    $cf : CauseFact(crossroad == crossroad, cause == cause)
end
```

Zašto autobus kasni? (jednostavna verzija)

```
query "whyBusDelaySimple"(String line, String crossroad, String cause)
    PublicTransportDelay(line == line, delayMinutes > 10)
    and $cf : CauseFact(crossroad == crossroad, cause == cause)
end
```

Algoritam za prevenciju ciklusa:

1. **Inicijalizacija**: `visitedCrossroads.clear()` briše sve prethodno posećene raskrsnice
2. **Provera posećenosti**: `!visitedCrossroads.contains($next)` sprečava vraćanje na već posećene raskrsnice
3. **Označavanje**: `visitedCrossroads.add($next)` dodaje raskrsnicu u set posećenih
4. **Rekurzija**: Query se poziva rekurzivno samo za neposećene raskrsnice

Primer toka dijagnoze sa prevencijom ciklusa:

1. Sistem detektuje kašnjenje autobusa na raskrsnici A
2. Query `findCauseOfDensity("A", result)` započinje pretragu
3. Proverava da li postoji direktan uzrok na A - ako ne, prelazi na rekurziju
4. Pretražuje povezane raskrsnice B, C, D:
    - Dodaje A u `visitedCrossroads` set
    - Za svaku povezanu raskrsnicu proverava `!visitedCrossroads.contains()`
    - Rekurzivno poziva `findCauseOfDensity("B", result)`

5. Ako se iz B pokuša vratiti na A, provera `!visitedCrossroads.contains("A")` vraća false
6. Ciklus je sprečen, pretraga se nastavlja kroz ostale neposećene raskrsnice
7. Kada se uzrok pronađe, insertuje se **CauseFact** objekat

Complex Event Processing (CEP)

Sistem implementira **2 napredna CEP pravila** za detekciju složenih vremenskih obrazaca sa validacijom crossroad referenci:

CEP Pravilo 1: Kompleksna Detekcija Saobraćajne Krize

Analizira kritične saobraćajne situacije kombinovanjem više koreliranih događaja preko različitih vremenskih prozora:

```
rule "Complex Traffic Crisis Detection"
when
    // Osnovno praćenje gustine saobraćaja preko 15-minutnog prozora
    $avgDensity : Number(doubleValue > 70) from accumulate(
        TrafficDensity($c : crossroad, $value : value) over window:time(15m) and
        exists Crossroad(id == $c),
        average($value)
    )

    // Brojanje kritičnih događaja u vremenskim prozorima
    $accidentCount : Number() from accumulate(
        Accident($c : crossroad) over window:time(20m) and
        exists Crossroad(id == $c),
        count(1)
    )

    $pedestrianCount : Number() from accumulate(
        PedestrianDetected($c : crossroad) over window:time(10m) and
        exists Crossroad(id == $c),
        count(1)
    )

    $busDelayCount : Number() from accumulate(
        PublicTransportDelay(delayMinutes > 10) over window:time(20m),
        count(1)
    )

    $weatherCount : Number() from accumulate(
        Weather(intensity == "high" || intensity == "heavy") over window:time(30m),
        count(1)
    )
```

```
    // Kalkulacija severity score-a krize
    eval(($accidentCount.intValue() * 3) +
         ($pedestrianCount.intValue() * 1) +
         ($busDelayCount.intValue() * 2) +
         ($weatherCount.intValue() * 2) >= 5)
then
    int crisisScore = ($accidentCount.intValue() * 3) +
                      ($pedestrianCount.intValue() * 1) +
                      ($busDelayCount.intValue() * 2) +
                      ($weatherCount.intValue() * 2);

    String severity = crisisScore >= 10 ? "CRITICAL" :
                      (crisisScore >= 7 ? "HIGH" : "MODERATE");

    // Eskalacija odgovora na osnovu ozbiljnosti
    if (crisisScore >= 10) {
        insert(new TrafficAction("SYSTEM", "activateEmergencyProtocol", 120));
        insert(new TrafficAction("SYSTEM", "blockNonEssentialTraffic", 60));
    } else if (crisisScore >= 7) {
        insert(new TrafficAction("SYSTEM", "activateRerouting", 90));
        insert(new TrafficAction("SYSTEM", "extendAllGreenLights", 30));
    } else {
        insert(new TrafficAction("SYSTEM", "adaptiveSignaling", 45));
    }
end
```

Komponente analize: - **Nesreće** (težina × 3): Najviši uticaj na saobraćaj - **Pešaci** (težina × 1): Uticaj na bezbednost - **Kašnjenje autobusa** (težina × 2): Uticaj na javni prevoz - **Loše vreme** (težina × 2): Uticaj na uslove vožnje

Nivoi odgovora: - **CRITICAL (10)**: Aktivacija emergency protokola + blokada saobraćaja - **HIGH (7-9)**: Rerouting + produžavanje zelenih signala - **MODERATE (5-6)**: Adaptivno signaliziranje

CEP Pravilo 2: Analiza Temporalnih Saobraćajnih Obrazaca

Analizira evoluciju saobraćajnih obrazaca i prediktivne trendove preko višestrukih vremenskih prozora:

```
rule "Temporal Traffic Pattern Analysis"
when
    // Analiza trenda saobraćaja preko višestrukih vremenskih prozora
    $currentAvg : Number(doubleValue > 60) from accumulate(
        TrafficDensity($c : crossroad, $value : value) over window:time(5m) and
        exists Crossroad(id == $c),
        average($value)
    )
```

```
    $previousAvg : Number() from accumulate(
        TrafficDensity($c : crossroad, $value : value) over window:time(5m, 10m) and
        exists Crossroad(id == $c),
        average($value)
    )

    // Korelacija sa masovnim događajima
    $eventDay : EventDay(expectedTraffic > 500)

    // Sistemski uticaj na raskrsnice
    $affectedCrossroads : Number(intValue >= 3) from accumulate(
        TrafficDensity($c : crossroad, value > 75) over window:time(15m) and
        exists Crossroad(id == $c),
        countDistinct($c)
    )

    // Kalkulacija pravca i magnitude trenda
    eval($currentAvg.doubleValue() - $previousAvg.doubleValue() > 15 ||
         $affectedCrossroads.intValue() >= 4)
then
    double trendChange = $currentAvg.doubleValue() - $previousAvg.doubleValue();
    boolean isWorsening = trendChange > 15;
    boolean isSystemWide = $affectedCrossroads.intValue() >= 4;

    String pattern;
    String action;
    int duration;

    if (isSystemWide && isWorsening) {
        pattern = "SYSTEM-WIDE CONGESTION SPREADING";
        action = "activateRegionalRerouting";
        duration = 180;
    } else if (isSystemWide) {
        pattern = "MULTI-CROSSROAD SATURATION";
        action = "coordinatedSignaling";
        duration = 120;
    } else if (isWorsening) {
        pattern = "RAPID CONGESTION BUILDUP";
        action = "preemptiveGreenExtension";
        duration = 60;
    } else {
        pattern = "MASS EVENT TRAFFIC DISTRIBUTION";
        action = "eventTrafficManagement";
        duration = $eventDay.getExpectedTraffic() > 2000 ? 240 : 120;
    }
```

```
    insert(new TrafficAction("SYSTEM", action, duration));
end
```

Analiza obrazaca: - **Sistemsko širenje gužve**: Regionalno preusmjeravanje (180s) - **Multi-crossroad saturacija**: Koordinirano signaliziranje (120s) - **Brzo nastajanje gužve**: Preventivno produžavanje zelenog (60s) - **Distribucija masovnog događaja**: Event traffic management (120-240s)

CEP Funkcionalnosti:

- **Vremenska okna**: 5min, 10min, 15min, 20min, 30min
- **Akumulacione funkcije**: count(), average(), countDistinct(), eval()
- **Korelacija događaja**: Kombinovanje 5+ različitih tipova događaja
- **Dinamičko skoriranje**: Weighted crisis scoring algoritam
- **Prediktivna analiza**: Trend detection preko sliding window-a
- **Validacija podataka**: Svi CEP queriji validiraju crossroad reference

Angular Frontend sa Real-time Monitoringom

Sistem koristi **Angular frontend** sa **PrimeNG** komponentama za intuitivni korisničko interfejs:

Ključne funkcionalnosti frontend-a:

1. Request Monitor - Praćenje poslanih zahteva

- Prikaz podataka koji se šalju backend sistemu
- Detaljan JSON prikaz faktova sa timestamps
- Kolapsibilni prikaz za detaljnu analizu zahteva

2. Response Dashboard - Centralni prikaz rezultata

- **Statistike**: Ukupan broj akcija, notifikacija, upita i grešaka validacije
- **TrafficAction panel**: Prikaz svih saobraćajnih akcija sa emoji ikonama
- **Notification panel**: Hierarchijski prikaz notifikacija sa prioritetima
- **Query Results**: Rezultati backward chaining upita
- **Validation Errors**: Jasno označene greške validacije sa ikonicom

3. Enhanced Error Handling

```
// Frontend automatski parsira validation errors
getValidationErrors(response: any[]): any[] {
    return response.filter(item =>
        item.query === 'ValidationError' || item.queryName === 'ValidationError'
    );
}
```

```
// Prioritizacija notifikacija
getNotificationPriority(message: string): 'high' | 'medium' | 'low' {
    if (message.includes('accident') || message.includes('block')) return 'high';
    if (message.includes('weather') || message.includes('delay')) return 'medium';
    return 'low';
}
```

4. Dynamic Fact Entry

- Interaktivni formulari za kreiranje različitih tipova faktova
- Dropdown selekcija za crossroad-e sa validacijom
- Real-time feedback o validaciji faktova

5. Visual Design

- **Priority styling**: Crveno za kritične, žuto za srednje, zeleno za informativne
- **Animirani indikatori**: Pulse efekat za nova obaveštenja

- **Responsive design**: Prilagođava se različitim veličinama ekrana
- **Dark/Light theme support**: Podrška za različite teme

Primer korisničkog interfejsa:

```
<!-- Request Monitor Section -->
<div class="request-monitor">
    <h3>  Request Monitor</h3>
    <div class="sent-data">
        <pre>{{formatRequestData(logEntry)}}</pre>
    </div>
</div>

<!-- Response Dashboard -->
<div class="response-dashboard">
    <div class="stats-overview">
        <div class="stat-card">
            <span class="stat-number">{{getTotalActions()}}</span>
            <span class="stat-label">Actions</span>
        </div>
        <div class="stat-card validation-errors" *ngIf="getTotalValidationErrors() > 0">
            <span class="stat-number error">{{getTotalValidationErrors()}}</span>
            <span class="stat-label">Validation Errors</span>
        </div>
    </div>

    <!-- Validation Errors Panel -->
    <div class="panel validation-errors" *ngIf="getAllValidationErrors().length > 0">
```

```
        <h4>  Validation Errors</h4>
        <div *ngFor="let error of getAllValidationErrors()" class="error-item">
            {{formatNotificationMessage(error)}}
        </div>
    </div>
</div>
```

Upravljanje stanjem i sesijom

Perzistentna KieSession

```
@Service
public class RuleExecutionService {
    private KieSession persistentSession;

    @PostConstruct
    public void initSession() {
        persistentSession = kieContainer.newKieSession("ksession-rules");
        persistentSession.setGlobal("visitedCrossroads", new HashSet<String>());
    }

    public List<QueryResultDto> execute(ExecuteRequest req) {
        // Validacija faktova pre inserovanja
        for (FactDto f : req.getFacts()) {
            ValidationResult validation = validationService.validateFactPayload(
                f.getType(), f.getPayload(), getCurrentSessionFacts()
            );

            if (!validation.isValid()) {
                // Dodaj validation error u rezultate
                validationErrors.add(new QueryResultDto("ValidationError",
                    "SYSTEM", "  " + validation.getErrorMessage()));
                continue;
            }

            // Ako je validacija prošla, insertuj fakt
            Object obj = mapFact(f.getType(), f.getPayload());
            if (obj != null) {
                persistentSession.insert(obj);
            }
        }

        persistentSession.fireAllRules();
        return collectResults();
    }
}
```

REST API Endpoints za upravljanje sesijom:

```
@RestController
@RequestMapping("/api/session")
public class SessionController {

    @GetMapping("/facts")
    public Map<String, Object> getSessionFacts() {
        return ruleExecutionService.getSessionState();
    }

    @GetMapping("/facts/count")
    public Map<String, Integer> getFactCounts() {
        return ruleExecutionService.getFactCounts();
    }

    @PostMapping("/clear")
    public ResponseEntity<String> clearSession() {
        ruleExecutionService.clearSession();
        return ResponseEntity.ok("Session cleared successfully");
    }
}
```

Tehnička specifikacija

Backend arhitektura:

- **Spring Boot 3.4.9** - Application framework
- **Drools 8.44.0.Final** - Rules engine sa CEP podrškom
- **Java 21** - Programming language
- **Maven** - Dependency management
- **Jackson** - JSON serialization

Frontend arhitektura:

- **Angular 18+** - Frontend framework
- **PrimeNG** - UI component library
- **SCSS** - Styling sa CSS custom properties
- **TypeScript** - Type-safe development
- **RxJS** - Reactive programming

Modularizacija:

- **model** - Domenski objekti (POJO klase)
- **kjar** - Drools knowledge base (DRL fajlovi)
- **service** - Spring Boot REST API
- **app** - Angular frontend aplikacija

Konfiguracija:

```
<!-- kmodule.xml -->
<kmodule xmlns="http://jboss.org/kie/6.0.0/kmodule">
    <kbase name="kbase-rules" packages="rules" stream-mode="true">
        <ksession name="ksession-rules" type="stateful" clock-type="realtime"/>
    </kbase>
</kmodule>
```

Zaključak

Implementiran je napredni sistem za upravljanje saobraćajem koji koristi:

Validaciju crossroad referenci u svim pravilima
Perzistentnu KieSession za kontinuirano praćenje stanja
Complex Event Processing sa 6 CEP pravila
Backward chaining za dijagnozu problema
Real-time Angular frontend sa intuitivnim UI
Comprehensive error handling sa detaljnim porukama
REST API za upravljanje sesijom
Type-safe architecture sa potpunom validacijom podataka

Sistem obezbeđuje **data integrity**, **real-time processing** i **user-friendly interface** za efikasno upravljanje gradskim saobraćajem.